Query Understanding:

WHATS SETFIT AND HOW IT WORKS:

Here's the clean explanation:

What is SetFit and how does it work?

SetFit stands for Sentence Transformer Fine-Tuning. It was developed by Hugging Face specifically to solve one problem — how do you train a high-accuracy text classifier when you only have a small amount of labeled data?

The problem with alternatives:

Fine-tuning BERT requires thousands of labeled examples to achieve reliable accuracy on domain-specific tasks. We had 354 labeled trade queries. Fine-tuning BERT on 354 examples would underfit significantly.

LLMs like GPT can classify text flexibly but they're slow, expensive per call, non-deterministic, and you can't measure their accuracy precisely per component. Wrong for a binary classification task that needs to run on every single search query in milliseconds.

How SetFit solves this:

SetFit works in two stages.

Stage 1 — Contrastive Fine-Tuning

From your labeled examples, SetFit automatically generates pairs:

- *Similar pairs — two BUY queries paired together, two SELL queries paired together*
- *Dissimilar pairs — one BUY query paired with one SELL query*

It then fine-tunes a Sentence Transformer using contrastive learning. The training signal is simple:

- *Pull similar pairs closer together in vector space*
- *Push dissimilar pairs further apart*

From 354 labeled examples, SetFit generates thousands of these pairs automatically. This is why it works with small datasets — it creates its own training signal through pairing.

After this stage, BUY-intent sentences cluster together in vector space. SELL-intent sentences cluster in a completely different region.

Stage 2 — Classification Head

A simple logistic regression classifier is trained on top of the fine-tuned embeddings. It draws a decision boundary between the BUY cluster and the SELL cluster.

At inference time:

- *New query comes in*
- *Sentence Transformer converts it to a vector*
- *Logistic regression checks which side of the boundary it falls on*
- *Returns BUY or SELL with a confidence score*

Why this works with only 354 examples:

The Sentence Transformer was pre-trained on hundreds of millions of sentence pairs from the internet. It already understands English semantics deeply. SetFit doesn't retrain it from scratch — it just fine-tunes it on your domain using contrastive pairs. The heavy lifting was already done during pre-training. You're just nudging it to understand your specific domain.

Why we chose SetFit:

- *354 labeled examples — SetFit is specifically designed for this scenario*
- *89.4% accuracy on our held-out test set — measurable, defensible*
- *27-48ms warm inference — fast enough for real-time search*
- *Deterministic — same query always returns same result, unlike LLMs*
- *We own the model — no API dependency, no per-query cost*

One line summary:

"SetFit achieves high accuracy on small labeled datasets by automatically generating contrastive training pairs from your examples and fine-tuning a Sentence Transformer — giving you BERT-level classification performance without needing thousands of labeled examples."

What does few-shot mean exactly?

In traditional machine learning, "more data = better model." Fine-tuning BERT for a new classification task typically requires 5,000 to 10,000 labeled examples to achieve reliable accuracy.

Few-shot learning flips this. Instead of learning from thousands of examples, the model learns from as few as 8 to 64 examples per class in research settings. SetFit is designed for this scenario — it achieves competitive accuracy with dramatically less labeled data by being clever about how it uses what little data it has.

In our case we had 354 labeled trade queries — 177 BUY and 177 SELL. That's few-shot territory. A standard fine-tuned BERT would underfit badly on this. SetFit achieved 89.4% accuracy.

Intent Detection:

*When a user types a query, the first thing our system does is figure out **what they want to do** — are they buying or selling?*

We use a fine-tuned ML model called SetFit for this. It's trained on real trade queries from our platform, so it understands context — not just keywords.

*But ML models make specific mistakes. For example, 'who sells sugar' — the model sees the word 'sells' and thinks it's a sell query. But actually the user is **looking for a seller**, so it's a buy query. So we have a pattern override layer that catches these exact cases and corrects them.*

The result is a single label — BUY or SELL — that drives everything downstream.

SetFit is a sentence classification model. We fine-tuned it on examples like:

"i want to buy sugar" → BUY
"looking for dextrose" → BUY
"i want to sell molasses" → SELL
"find buyers for urea" → SELL

So it learned to associate sentence patterns with BUY or SELL.

At inference time we just do:

python
```
pred = self._intent_model.predict([query])[0]
```

We pass the raw query string in, it returns `0` or `1`, we map that to `BUY` or `SELL`. That's it — one function call, no logic, the model does everything internally.

What's happening inside SetFit when you call predict:

1. Query gets converted to a vector embedding (numerical representation of meaning)
2. That vector gets passed to a small classifier head
3. Classifier outputs 0 (BUY) or 1 (SELL)

—

For intent detection we use a fine-tuned SetFit model — trained on real trade queries from our platform. We pass the raw query in, it returns BUY or SELL in milliseconds based on sentence context, not just keywords.

But ML models have blind spots. 'Who sells sugar' — the model sees 'sells' and says SELL. But the user is looking for a seller, so it should be BUY. So we have a regex override layer that catches these specific cases and corrects the prediction.

So the final intent is always reliable — ML handles the general case, regex handles the edge cases

Why SetFit over the alternatives:

vs Regex: We tried regex in v1. It has zero semantic understanding. If a trader types "our warehouse is running low on raw sugar" — no intent keyword like "buy" exists. Regex fails completely. SetFit understands the meaning — that phrase geometrically aligns with BUY intent in vector space even without explicit trigger words.

vs Fine-tuned BERT: BERT needs thousands of labeled examples to reach reliable accuracy on domain-specific tasks. We had 354 trade queries. Fine-tuning BERT on 354 examples would underfit badly. SetFit's contrastive learning generates thousands of training pairs from those 354 examples automatically — achieving the same accuracy BERT would need 10x the data to reach.

vs LLMs: GPT can classify intent accurately but takes 2-4 seconds per API call and costs money per token. You cannot run a real-time search bar on a 3 second delay. SetFit runs locally in 27-48ms with zero API cost. Same semantic reasoning, 100x faster, zero ongoing cost.

vs Zero-shot classifiers: Generic zero-shot models like BART-MNLI struggle with niche B2B agricultural trade language. They've never seen "offloading molasses stock" or "sourcing dextrose monohydrate." SetFit lets us inject our specific domain knowledge through those 354 labeled examples — making it hyper-accurate for our use case.

One line that ties it all together:

"SetFit is the only architecture that gives us LLM-level semantic understanding, runs locally in milliseconds, and works reliably with the small labeled dataset we had. Every alternative fails on at least one of those three constraints

1. "Few-Shot" Efficiency (No Big Data Required)

If you want to train a traditional AI model (like a standard BERT classifier) to understand the difference between a Buyer and a Seller, you usually need to feed it **thousands** of manually labeled sentences. Hand-labeling thousands of trade queries is incredibly expensive and time-consuming.
SetFit is famous for being a **Few-Shot** learner. It can achieve state-of-the-art accuracy by looking at as few as **8 to 16 examples** per category. This allowed us to create a highly specialized, hyper-accurate B2B trade intent model in an afternoon, without needing a massive data science budget or thousands of rows of training data.

2. Contrastive Learning (Understanding Extreme Nuance)

In global trade, the difference between a Buyer and a Seller is often just a single word.

- *Sentence A:* "We **have** a massive surplus of sugar." (SELL)
- *Sentence B:* "We **need** a massive surplus of sugar." (BUY)

Traditional models often get confused because 90% of the words in those two sentences are identical. SetFit uses **Contrastive Learning**. During its brief training, it looks at pairs of sentences and mathematically pushes them apart in vector space. It learns to heavily polarize the mathematical "vibe" of the words *"have"* versus *"need"*. Because it maps sentences geometrically, it can instantly spot the microscopic nuances that change the entire intent of a query.

3. Handles Conversational Ambiguity

If a user searches: *"We are a large agricultural farm based in Brazil looking to expand our market reach."* There is no "buy" or "sell" trigger word in that sentence. But SetFit converts that entire sentence into a mathematical embedding. It realizes that the semantic concepts of *"farm"* and *"expand market reach"* geometrically align with the `SELL` pole. It correctly classifies the user as a Seller, whereas a strict Regex rule would have crashed.

4. Lightning Fast & CPU Friendly

Just like GLiNER, relying on a massive external LLM (like GPT-4) to classify intent is too slow and too expensive for a live search bar. SetFit models are incredibly tiny (often based on MiniLM, containing only ~22 million parameters). It executes entirely locally on standard CPU backend servers in a fraction of a millisecond.

Summary: We use SetFit because it gives us the deep semantic reasoning of a massive language model, but it only required a handful of examples to train, and it runs locally at lightning speed for free!

WHATS GLINER AND HOW IT WORKS:

Here's the simplified version:

What is GLiNER and how does it work?

GLiNER stands for Generalist Model for Named Entity Recognition. It bridges the gap between two extremes — traditional NER models that are fast but rigid, and LLMs that are flexible but slow and expensive.

The problem with alternatives:

Traditional NER models like spaCy are trained on fixed entity types — person, location, organization. To add a new entity type like "commodity" you need to retrain on thousands of labeled examples. Rigid and expensive to maintain.

LLMs like GPT can extract any entity type flexibly, but they generate text one token at a time — sequential and slow. 2-5 seconds per query. Expensive per call. Completely wrong for a real-time search bar.

How GLiNER solves this:

Instead of generating text sequentially like an LLM, GLiNER frames entity extraction as a parallel matching problem.

Here's the step by step:

Step 1 — Unified input GLiNER takes the entity labels and the query together as one input: `[ENT] product [ENT] country [SEP] buy dextrose from brazil`

Step 2 — Bidirectional processing It uses a compact bidirectional transformer (DeBERTa) — reads the entire sentence simultaneously in both directions. This gives it full sentence context, not just left-to-right like sequential models.

Step 3 — Latent space matching It maps both the entity labels and text spans into the same mathematical space. Then calculates similarity scores between each text span and each entity label using dot product similarity.

Step 4 — Extraction Any span scoring above 0.5 similarity with an entity label gets extracted. "dextrose" scores high for "product". "brazil" scores high for "country". Both extracted in one pass.

Why this matters for us:

- Zero-shot — we just pass `["product", "country"]` at runtime. No retraining needed when new commodities appear
- Single pass — extracts product AND country simultaneously
- Runs locally — 90 million parameters, runs on CPU, milliseconds per query, zero API cost
- Despite being 140x smaller than some LLMs it outperforms them on zero-shot NER benchmarks

---

One line summary:

"GLiNER gives us LLM-level flexibility for extracting any entity type, at traditional NER model speed and cost — by framing extraction as a parallel matching problem rather than sequential text generation."

–

Product Extraction Pipeline

We extract the product from any query — clean, typo-ridden, or grammatically broken — through four steps.

Step 1 — GLiNER extracts the product span

GLiNER reads the raw query using bidirectional transformers. It understands grammatical context — it knows the noun after "buy" or "need" is the commodity without needing to know what that commodity is. For `"buy dextrse anhydros from china"` it extracts `"dextrse anhydros"`.

Step 2 — Validation

We check if GLiNER's extraction makes sense. If it accidentally extracted a verb like "exporting" or a country name like "brazil" — we discard it and fall back to Step 3. If it's clean — we skip Step 3 entirely.

Step 3 — Fallback: Stop-word stripping

Only activates if GLiNER failed. We remove all known trade words — "buy", "sell", "from", "urgent", "cheap" — from the raw query. Whatever survives is the product. This guarantees we always have something to work with.

Step 4 — Spell correction

We run RapidFuzz against our product catalog — all SubCategory and Item names loaded from the database into memory at server startup. For clean words it returns the same word unchanged. For typos it corrects them:

- "suggar" → "Sugar"
- "dextrse anhydros" → "Dextrose Anhydrous"

The corrected keyword then maps to our database taxonomy and powers the search

Country Extraction

Country extraction works in three steps. First GLiNER reads the sentence and suggests a country span based on grammatical context. We then validate that suggestion against our country catalog built from pycountry — 249 countries with full names, alpha codes, and common names. If it's an exact match, done. If it's a typo or shorthand like 'brzail' or 'pak', we run RapidFuzz fuzzy matching to find the closest real country. If GLiNER finds nothing at all, we fall back to regex — scanning for preposition patterns like 'from X' or 'in X' and running the same validation on whatever follow.

Here is your ultimate "Cheat Sheet" for the entire ZaraiLink NLU Engine.

There are **4 Layers for Country Extraction**, and **5 Layers for Product Extraction**.

🌍 Country Extraction (4 Layers)

The Country pipeline is a "waterfall". It drops down layer-by-layer until it successfully finds the country, and then it stops.

Layer 1: The AI Suggestion (GLiNER)
*** **What it does:** Reads the grammar of the sentence (e.g., *"buy from X"*) and guesses the raw text of the location.**
*** **Next Step:** Sends the guess to Layer 2 for validation.**

Layer 2: The Validation Gauntlet (`_resolve_country`)
*** **What it does:** The ultimate security checkpoint. It forces any guess to pass 4 checks:**
1. **Noise Filter: Destroys words like "global".**
2. **Slang Dictionary: Translates "uae" to "United Arab Emirates".**
3. **Exact Match: Capitalizes "pakistan" to "Pakistan".**
4. **RapidFuzz (85%): Mathematically fixes severe typos (e.g. "brazl" -> "Brazil").**

Layer 3: The Grammar Fallback (Regex)
*** **What it does:** Only activates if Layer 1 & 2 failed. It blindly hunts the text for prepositions (`from`, `frm`, `in`). If it finds one, it extracts the next word and throws it back up to Layer 2.**

Layer 4: The Brute Force Scan
*** **What it does:** Only activates if there were no prepositions in the sentence (Layer 3 failed). It breaks the sentence into single words and checks if any word over 3 letters perfectly matches a country in the database.**

****

 # 📦 Product Extraction (5 Layers)

The Product pipeline is an "assembly line". Every extraction must pass through the gauntlet to be cleaned and finalized.

Layer 1: The AI Suggestion (GLiNER)

*** **What it does:** Reads the raw query and uses grammatical context to extract the suspected commodity (e.g., guessing `"export suggar"`).**

Layer 2: The Rejection Shields
*** **What it does:** Defends against AI hallucinations.**
1. **Verb Shield: Did the AI extract a verb? (e.g. `"export"`). If yes, destroy the guess.**
2. **Country Shield: Did the AI extract a country? (e.g. `"brazl"`). If yes, destroy the guess.**

Layer 3: The Complete Failure Recovery (Stop-Word Stripper)
*** **What it does:** Only activates if Layer 2 destroyed the AI's guess (meaning we currently have no product). It uses subtraction. It takes the raw user query and brutally deletes every known trade word (`buy`, `sell`, `cheap`, `urgent`). Whatever words survive the deletion become the product.**

Layer 4: The Cleaning Sweepers
*** **What it does:** Takes the product (either from Layer 1 or Layer 3) and cleans off any lingering junk.**
1. **Preposition Sweeper: Uses Regex to erase dangling bridge words (e.g., removing `"from"` from `"sugar from"`).**
2. **Noise Stripper: Uses RapidFuzz to mathematically erase slang (e.g., removing `"urgent"`).**

Layer 5: The Catalog Spell Checker (Pass 5.6)
*** **What it does:** The final step. It takes the isolated product word and runs RapidFuzz math against the official Postgres database catalog at a `60%` threshold. If the user's typo (`"suggar"`) mathematically matches the DB (`"Sugar"`), it fixes the spelling so the search executes flawlessly.**

JUSTIFICATION:

Here's what to say — structured as a proper justification showing you evaluated alternatives:

Why not traditional NER models (spaCy, NLTK, Stanford NER)?

Traditional NER models are trained on fixed entity types — person, organization, location, date. They don't know what a "commodity" or "trade product" is. To use spaCy for product extraction you'd need to retrain it on domain-specific labeled data — thousands of annotated examples of trade queries with product spans marked. We had 354 labeled examples for SetFit intent classification. We didn't have thousands of NER-annotated examples. And even if we retrained, the model would only recognize products it saw during training. A new commodity like a specialized chemical compound would fail completely.

"Traditional NER models require domain-specific retraining and can't generalize to unseen product names. That's a fundamental limitation for B2B trade where thousands of obscure commodities exist."

Why not rule-based approaches (Regex, FlashText, keyword lists)?

We tried this in v1. The problem is you're essentially predicting every possible way a human might type a query. "buy sugar", "need sugar", "source sugar", "looking for sugar", "sugar suppliers needed" — each requires a different rule. It becomes unmaintainable and still fails on anything you didn't explicitly anticipate. A new product, a new phrasing, a typo — all break the rules.

"Rule-based extraction requires anticipating every possible query pattern. Real trader language is too variable and unpredictable for this approach to scale."

Why not an LLM (GPT, Claude, Gemini)?

LLMs understand language extremely well — they would extract products and countries reliably. But there are three problems. First, latency — every query requires an API call taking 2-5 seconds. A search bar that takes 5 seconds per keystroke is unusable. Second, cost — at scale, paying per token for every search query is commercially unviable. Third, it's a black box — you can't measure extraction accuracy per component, you can't debug why a specific query failed, you can't make targeted improvements.

"LLMs are powerful but wrong for this use case — too slow, too expensive, and too opaque for a component where we need measurable, debuggable, sub-100ms performance."

Why GLiNER?

GLiNER is zero-shot — it generalizes to entity types it was never trained on. We just pass labels like `["product", "country"]` and it extracts those spans based on grammatical context, not memorized dictionaries. It runs locally — no API, no cost, milliseconds per query. It handles both product and country extraction in a single pass. And critically — it doesn't need to know what "Dextrose Monohydrate" or "Xylophantium" is. It knows the noun after "buy" is the product because it understands English sentence structure.

"GLiNER gives us LLM-level grammatical understanding at local inference speed with zero API cost. It's the exact right tool for named entity extraction from short noisy queries."

One line that ties it all together:

"We needed a model that generalizes to unseen commodities, runs locally in milliseconds, requires no domain retraining, and handles both product and country extraction in one pass. GLiNER is the only model that satisfies all four constraints simultaneously."

** PRODUCT EXTRACTION **

How we implemented GLiNER for product extraction:

Step 1 — Loading the model

We loaded a pretrained GLiNER model (`urchade/gliner_base`) from HuggingFace. We cache it as a class attribute so it loads once and stays in memory for all subsequent queries. No reloading per request — first query pays the loading cost, every query after is instant.

Step 2 — Telling GLiNER what to look for

We gave GLiNER 4 product labels intentionally: `product`, `commodity`, `agricultural product`, `trade good`

Why 4 and not just 1?

Because GLiNER is zero-shot — it matches spans based on the semantic meaning of the label name itself. Different query styles trigger different labels:

- `"buy dextrose"` → tagged as `product`
- `"buy wheat grain"` → tagged as `agricultural product`
- `"buy PVC resin"` → tagged as `commodity`
- `"buy basmati rice bulk"` → tagged as `trade good`

If we only used `product`, we'd miss the other three cases entirely. More labels = more chances to catch the product regardless of how the user describes it.

Step 3 — Running GLiNER

Two important decisions here:

Raw query, no cleaning — We pass the query before any country stripping or cleaning. This gives GLiNER full sentence context which improves accuracy. If we cleaned first, GLiNER would lose context and make worse predictions.

Threshold = 0.3 — GLiNER scores every span from 0 to 1 based on how confident it is. Threshold is the minimum score to accept a span. We set it low at 0.3 intentionally — better to extract something with low confidence and reject it downstream through our fixes, than to miss it entirely. On typo-heavy queries like `"suggar cheep frm brazl"` GLiNER isn't very confident, but at 0.3 it still picks something up. Our fixes then clean it.

Step 4 — Checking all 4 labels in priority order

First match wins. We check `product` first as it's the most specific, falling through to broader labels only if needed.

Issues GLiNER had and fixes we implemented:

Issue 1 — Tagged action verbs as product

What happened: For a query like `"export dextrose from china"`, GLiNER sees `export dextrose` as one span and tags the whole thing as the product. This is because GLiNER looks at surrounding context — `export` sits right next to a commodity word so it gets swept into the span boundary.

What breaks: The product becomes `"export dextrose"` which doesn't match anything in the DB. Search returns zero results.

Fix: After GLiNER returns a span, we check if it starts with or equals any known action verb — import, export, buy, sell, buying, selling, importing, exporting. If it does, we reject the span entirely and fall to the next extraction method.

```
_action_verbs = ("import", "export", "buy", "sell", "buying", "selling")
if raw_product in _action_verbs or raw_product.startswith(tuple(f"{v} " for v in _action_verbs)):
    product_keyword = None
```

Issue 2 — Tagged country as product

What happened: For a query like `"sugar from brazl"`, GLiNER sometimes tags `brazl` as a product span because it's an unfamiliar word and GLiNER has low confidence about what it is. It scores it higher as `product` than `country` because the spelling is too broken for GLiNER to recognize it as a location.

What breaks: Product becomes `"brazl"`, country becomes None. Search looks for a product called brazl in the DB — finds nothing.

Fix: After accepting a GLiNER product span, we run it through our country resolver at a low cutoff of 60. If it resolves as any country, we reject it as a product.

```
_country_check = self._resolve_country(raw_product, cutoff=60.0)
if _country_check:
    product_keyword = None
```

Issue 3 — Included noise words in span

What happened: For a query like `"where can i find suppliers of dextrose"`, GLiNER tags `can dextrose` as the product span. This happens because `can` sits adjacent to

`dextrose` and GLiNER's span boundary detection includes it. The model sees `can dextrose` as one chunk because there's no strong separator between them in the sentence structure.

What breaks: Product becomes `"can dextrose"` which doesn't exist in the DB. Also `can` matches Canada's alpha-3 code, causing country false positives.

Fix: After extraction we run a regex cleanup that strips known noise words from the product span:

```
_PREP_RESIDUAL_RE = re.compile(
    r'\b(from|frm|in|at|can|get|find|suppliers?|of)\b',
    re.IGNORECASE,
)
product_keyword = _PREP_RESIDUAL_RE.sub(' ', product_keyword).strip()
# "can dextrose" → "dextrose"
```

Issue 4 — Missed product entirely on typo-heavy queries

What happened: For a query like `"suggar cheep frm brazl"`, every word is misspelled. GLiNER's confidence on all spans drops below our threshold of 0.3 because it can't confidently identify any span as a known product type. It returns nothing.

What breaks: No product extracted, search returns broad/empty results.

Fix: We built a fallback chain — GLiNER failing is not the end:

```
GLiNER fails
  → Try LLM (OpenRouter/DeepSeek) — only if price numbers present
    → Stop-word stripper — removes all known trade words
      (buy, sell, from, cheap, frm, urgent etc.)
      and returns whatever's left → "suggar"
```

The stop-word stripper always returns something as long as there's a non-trade word in the query. Product extraction never returns empty.

Issue 5 — Extracted product was a typo

What happened: GLiNER or the fallback correctly identifies `suggar` as the product — but `suggar` doesn't exist in our database. Neither does `dextrse` or `sugr`. The extraction was correct but the spelling was wrong.

What breaks: DB query finds no matching subcategory. Disambiguation returns wrong variants or no results.

Fix: RapidFuzz spell correction against our actual product catalog. We load all product and subcategory names from the DB once and cache them on the engine instance. Then every extracted single-word product gets fuzzy matched:

```
_sc_match = process.extractOne(
    product_keyword,
    self._product_catalog_cache,
    scorer=fuzz.ratio,
    score_cutoff=60,
)
if _sc_match:
    product_keyword = _sc_match[0]
# "suggar" → "Sugar"
# "dextrse" → "Dextrose"
# "sugr" → "Sugar"
```

Score cutoff 60 means we only accept corrections that are at least 60% similar — prevents random words from matching unrelated products.

End result: No matter what query comes in — casual language, typos, person names, noise words — we always end up with the correct product name that exists in our database.

what are we doing

what is Gliner and how does it work

why did you choose Gliner and not Spacy NER or any other model for product extraction?

GLINER-

Its a:

A zero-shot Named Entity Recognition (NER) model is an AI system that identifies and

classifies entities (e.g., names, locations, custom terms) in text without requiring prior, specialized training data for those specific labels. These models leverage pre-trained knowledge to recognize unseen entities on the fly, reducing the need for costly, manual, domain-specific labeling

GLiNER is basically a lightweight transformer-based entity extraction system.

For our search engine, we used it because we needed flexible entity extraction without depending on huge LLMs.

The way it works is:
first, we provide the entity categories we care about — for example:

- product
- Trade commodity
- Agricultural product
- product

Then GLiNER combines those labels with the actual user query and passes everything through a bidirectional transformer encoder like BERT.

Because it's bidirectional, the model understands the entire sentence context at once instead of generating text word-by-word like GPT.

After encoding the query, GLiNER checks different chunks of text and determines which chunks match which entity labels.

So for a query like:
'import dextrose monohydrate from china'

it can identify:

- 'dextrose monohydrate' → product
- 'china' → country

The important part is that the labels are dynamic. We don't need to retrain the model every time we introduce a new entity type.

That made it useful for our search engine because user queries are very inconsistent and domain-specific. Instead of hardcoding regex rules for every possible case, GLiNER gives us semantic entity extraction using a compact model that runs much faster and cheaper than an LLM

Why We Use GLiNER and How It Works

GLiNER (Generalist Lightweight INformation Extraction) is a state-of-the-art, transformer-based Named Entity Recognition (NER) model. In the ZaraiLink search architecture, it serves as the primary engine for extracting critical trade parameters from messy, natural language queries.

We chose GLiNER because B2B trade search requires highly flexible entity extraction without the massive latency and compute costs of a Large Language Model (LLM), and without the rigid limitations of traditional dictionary-based NER systems.

How GLiNER Works: The Mechanics

Traditional NER models must be heavily fine-tuned on vast datasets to memorize specific words (e.g., training a model to memorize that "dextrose" is a chemical). GLiNER takes a fundamentally different, **zero-shot** approach:

1. **Dynamic Prompting:** Instead of relying on a hardcoded vocabulary, we pass the user's raw query to GLiNER alongside a list of dynamic entity labels we care about for that specific search context (e.g., `["product", "commodity", "country", "trade term"]`).
2. **Bidirectional Contextual Encoding:** GLiNER processes both the text and the labels through a bidirectional transformer encoder (like BERT). Unlike generative models (like GPT) that read word-by-word sequentially, GLiNER evaluates the *entire* sentence at once. It understands the grammatical structure and the surrounding context of the words.
3. **Span Prediction:** After encoding the text, GLiNER evaluates different chunks (spans) of the sentence and computes the semantic similarity between those text chunks and the labels we provided.

Because of this semantic matching, if a user queries:

"import dextrose monohydrate from china"

GLiNER doesn't need to know what *"dextrose monohydrate"* actually is. It simply understands the grammatical pattern (*import [X] from [Y]*) and confidently maps the span `"dextrose monohydrate"` to the `product` label, and `"china"` to the `country` label.

Why We Use GLiNER at ZaraiLink

User queries in global trade are incredibly inconsistent, heavily abbreviated, and highly domain-specific.

1. **Future-Proof against Novel Vocabulary:** Global trade involves tens of thousands of obscure agricultural goods and manufactured chemicals. A traditional model will fail when a user searches for a brand new commodity it has never seen before. GLiNER extracts products purely based on their sentence context, meaning we never have to retrain the model when a new product hits the market.

2. **Eliminating Regex Hell:** Trying to hardcode regular expressions to catch every possible way a buyer might phrase a search is impossible and unmaintainable. GLiNER replaces thousands of lines of fragile parsing logic with robust, semantic extraction.
3. **High Speed, Low Compute:** While an LLM could perform this extraction, calling an LLM takes 3 to 5 seconds and costs money per token. GLiNER is a compact model that runs entirely locally, delivering LLM-quality extraction in milliseconds at a fraction of the compute cost.

6:52 PM

encoding query means- first tokenizer splits query into small chunks

each token gets an id

that id is converted into vectors

and hence encoded.

this passes through transformer layers and the model builds conetxtual understanding . and its understanding changes depening on surrounding words.

Gliner does not understand products it understand contextual patterns. This is exactly what transformers learn.

In NLP theres a famous principle- You shall know a word by the company it keeps.

Words derive meaning from surrounding words.

Encoding a query means the text is first tokenized into smaller chunks, each token is mapped to an ID, and then converted into vectors using embeddings.
These vectors are passed through transformer layers, where self-attention allows each token to interact with all others.
This produces contextual representations, meaning the embedding of each word changes depending on the surrounding words and overall sentence meaning.

Issues Gliner caused and how we fixed them:

Building a Robust Product Extraction Pipeline

While GLiNER is an incredibly powerful AI model for extracting entities based on grammatical context, it is not flawless. Because B2B trade queries are often riddled with typos, slang, and broken grammar, GLiNER occasionally fails in very specific, predictable ways.
To guarantee that the search engine always extracts the correct product, we built a highly robust pipeline consisting of 5 specific "Safety Nets." Here is a detailed breakdown of the exact issues GLiNER faced and how the engine fixes them.

Issue 1: Tagging Action Verbs as Products

WARNING

What Happened: For a query like *"export dextrose from china"*, GLiNER sometimes sees "export dextrose" as a single span and tags the whole thing as the product. This happens because GLiNER looks at surrounding context—the word "export" sits right next to a commodity word, so the AI sweeps it into the span boundary.

What Breaks: The product becomes `"export dextrose"`, which doesn't match anything in the database. The search returns zero results.
The Fix (The Verb Shield): After GLiNER returns a span, the engine explicitly checks if it starts with or equals any known action verb (e.g., import, export, buy, sell). If it does, we reject the AI's guess entirely and fall back to the next extraction method.
python

```
_action_verbs = ("import", "export", "buy", "sell", "buying", "selling")
if raw_product in _action_verbs or raw_product.startswith(tuple(f"{v} "
for v in _action_verbs)):
    product_keyword = None  # Reject the AI's span
```

Issue 2: Tagging a Country as a Product

WARNING

What Happened: For a query like *"sugar from brazl"*, GLiNER sometimes tags "brazl" as a product span. Because the word is misspelled and unfamiliar, GLiNER has low confidence. It accidentally scores it higher as a product than a country because the spelling is too broken for it to recognize it as a location.

What Breaks: The Product becomes `"brazl"`, and the Country becomes `None`. The search looks for a physical commodity called "brazl" and finds nothing.

The Fix (The Country Cross-Check): After accepting a GLiNER product span, we run that span through our Country Resolver at a strict cutoff of 60%. If the extracted "product" mathematically resolves to any known country, we immediately reject it.

```python
_country_check = self._resolve_country(raw_product, cutoff=60.0)
if _country_check:
    product_keyword = None  # Reject because it's actually a location
```

Issue 3: Including Noise Words in the Span

WARNING

What Happened: For a query like *"where can i find suppliers of dextrose"*, GLiNER tags "can dextrose" as the product span. This happens because "can" sits adjacent to "dextrose", and without a strong separator, GLiNER's boundary detection groups them together.

What Breaks: The Product becomes `"can dextrose"`, which doesn't exist. Even worse, the word "can" matches Canada's alpha-3 country code, causing massive false positives.

The Fix (The Residual Sweeper): After extraction, we run a regular expression cleanup that surgically strips known conversational noise words from the product span.

```python
_PREP_RESIDUAL_RE = re.compile(
    r'\b(from|frm|in|at|can|get|find|suppliers?|of)\b',
    re.IGNORECASE,
)
product_keyword = _PREP_RESIDUAL_RE.sub(' ', product_keyword).strip()
# "can dextrose" → "dextrose"
```

Issue 4: Missing the Product Entirely on Messy Queries

WARNING

What Happened: For a query like *"suggar cheep frm brazl"*, every single word is misspelled. Because it can't confidently identify anything, GLiNER's confidence drops below our threshold, and it returns absolutely nothing.

What Breaks: No product is extracted, causing the search to return completely unfiltered, broad results.

The Fix (The Fallback Chain): GLiNER failing is not a dead end. The engine cascades through a series of simpler, deterministic fallbacks.

1. **GLiNER Fails** →
2. **Try LLM API** (Only if complex numbers/prices are present) →
3. **Stop-Word Stripper** (The ultimate safety net)

The Stop-Word stripper doesn't use AI. It takes the raw query and simply deletes every known trade word (`buy`, `sell`, `from`, `cheap`, `frm`, `urgent`). It returns whatever words survive the massacre (e.g., `"suggar"`). The stripper always returns *something*, guaranteeing product extraction never returns empty.

Issue 5: The Extracted Product was a Typo

WARNING

What Happened: The fallback chain correctly identifies `"suggar"` as the product. However, `"suggar"` does not exist in the database schema. Neither does `"dextrse"` or `"sugr"`. The extraction was successful, but the spelling was fatal.

What Breaks: The database queries fail to find matching subcategories. Disambiguation returns wrong variants or no results.
The Fix (Pass 5.6 - Catalog Spell Correction): We use `RapidFuzz` for mathematical spell correction against our *actual* product catalog. We load all official product names from the database once and cache them. Then, every extracted single-word product is fuzzy-matched against the official list.

```python
_sc_match = process.extractOne(
    product_keyword,
    self._product_catalog_cache,
    scorer=fuzz.ratio,
    score_cutoff=60,
)
if _sc_match:
    product_keyword = _sc_match[0]
```

- `"suggar"` → `"Sugar"`
- `"dextrse"` → `"Dextrose"`
- `"sugr"` → `"Sugar"`

TIP

The score cutoff is strictly set to `60`. This means we only accept corrections that are at least 60% mathematically similar, preventing completely random words from mapping to unrelated products.

After the text is passed through a bidirectional transformer, GLiNER turns every token into contextual vector representations, meaning each word now carries information about the full sentence. It then forms candidate “spans,” which are continuous chunks of the sentence (like “Bill Gates” or “Microsoft”), by combining the vectors of tokens within those chunks into a single representation. At the same time, the entity labels you provide (like “person” or “organization”) are also converted into vectors in the same semantic space. Once both spans and labels are represented as vectors, GLiNER compares them using similarity scoring. In simple terms, it

checks how close the meaning of a text chunk is to the meaning of a label. If a span vector is very similar to a label vector, the model assigns that label to that span. This is how it decides that “Bill Gates” matches “person” and “Microsoft” matches “organization.”

JUSTIFICATION:

Architectural Justification: Why GLiNER for Product Extraction?

When designing a search engine for B2B global trade, extracting the exact commodity from a natural language query is the most complex engineering challenge. B2B queries are plagued by typos, heavy abbreviations, non-standard grammar, and a vocabulary spanning tens of thousands of obscure agricultural and chemical terms.
To solve this, we evaluated five different approaches to Named Entity Recognition (NER). Below is the justification for why **GLiNER (Generalist Lightweight INformation Extraction)** was selected as the optimal architecture for our production environment.

The Alternative Approaches (And Why They Failed)

Before arriving at GLiNER, we evaluated the standard industry approaches for product extraction.

1. Dictionary / Gazetteer Lookup (e.g., FlashText, Aho-Corasick)

- **How it works:** You load your entire database of products into memory. If a word in the user's query exactly matches a word in the dictionary, it is extracted.
- **Why we rejected it:** It is entirely rigid. If a user types a synonym, an abbreviation (e.g., "dex" for Dextrose), a severe typo, or searches for a novel commodity that hasn't been added to our database yet, the engine returns zero results. It cannot handle the "long-tail" of conversational search.

2. Traditional Supervised NER (e.g., spaCy, BERT-NER)

- **How it works:** You train an NLP model to recognize products by manually highlighting the "product" in thousands of sample sentences.
- **Why we rejected it:** Data starvation and model decay. Traditional NER requires massive labeled datasets to be effective. Worse, they effectively "memorize" the training vocabulary. If a new chemical compound hits the market tomorrow, a traditional spaCy model will likely ignore it because it wasn't in the training data. Maintaining and retraining the model every month is an operational nightmare.

3. Statistical Keyword Extraction (e.g., KeyBERT, TF-IDF)

- **How it works:** Uses algorithms to find the most "statistically important" word in the sentence. (This was actually our legacy system).
- **Why we rejected it:** Keyword extractors look for *importance*, not *semantics*. In a query like *"urgently looking for buyers"*, an algorithm like KeyBERT often extracts "urgently" because it is a statistically rare and heavily weighted word. It lacks the semantic understanding of what a physical "product" actually is.

4. Large Language Models (e.g., GPT-4, Claude)

- **How it works:** You send the query to an API and prompt the LLM: *"Extract the product from this sentence."*
- **Why we rejected it:** Latency and Cost. While LLMs have incredible extraction accuracy, calling an external API takes 2 to 5 seconds per query. This creates an unacceptably slow user experience for a core search bar. Furthermore, paying per-token for every single search query makes scaling financially unviable.

Why GLiNER is the Superior Choice

GLiNER solves the trade-offs of every approach listed above. It provides the **reasoning of an LLM** with the **speed and cost of a local model.**
Here are the specific justifications for choosing GLiNER:

1. Zero-Shot Contextual Understanding

GLiNER does not memorize dictionaries. It uses a **Bidirectional Transformer** to read the entire sentence at once, focusing heavily on grammatical structure. When a user types *"I want to buy Xylophantium from China"*, GLiNER does not need to know what "Xylophantium" is. It understands the linguistic pattern: *"buy [X] from [Y]"*. Because it understands the grammatical context, it can perfectly extract completely novel, unseen, or highly misspelled commodities.

2. Dynamic Labeling (No Retraining Required)

With traditional models, if you decide you want to extract "Chemicals" instead of "Products", you have to re-annotate your dataset and retrain the model. GLiNER uses **dynamic prompting**. We pass the text along with a list of labels at inference time: `["product", "commodity", "agricultural product"]`. GLiNER dynamically checks the text against these semantic concepts. If our business requirements change, we simply update the text labels in the code—zero retraining required.

3. High Speed, Local Compute

Because GLiNER is a "lightweight" model (around 300M parameters compared to GPT-4's trillions), it runs entirely locally. It does not require massive GPU clusters. It executes in milliseconds on standard backend infrastructure, ensuring search results are returned instantly without racking up external API costs.

4. Seamless Fallback Integration

While GLiNER handles the heavy lifting of semantic understanding, its lightweight nature allows us to easily wrap it in deterministic safety nets. If GLiNER extracts a span containing a typo, our backend seamlessly passes that span to RapidFuzz for mathematical correction against our database. It plays perfectly within a hybrid AI/Deterministic architecture.

Conclusion

GLiNER is the only approach that successfully balances **accuracy on unseen vocabulary** with **sub-second latency**. It allows ZaraiLink to process messy, conversational global trade queries natively, future-proofing the search engine against novel commodities without requiring constant, expensive model retraining.

Yeah so for product extraction, we use GLiNER as a semantic entity extractor.

Basically, when a user sends a query, we don't rely on regex or fixed rules. Instead, we pass the query into GLiNER along with a dynamic list of entity types like product, country, company, etc.

GLiNER uses a transformer encoder to understand the full sentence context, then it breaks the text into candidate spans — basically phrases — and checks which of those spans semantically match the label 'product'.

So for example, in a query like 'import dextrose monohydrate from china', it identifies 'dextrose monohydrate' as a product and 'china' as a country based on similarity in embedding space, not keywords.

The key idea is that it's doing span-level semantic matching instead of rule-based parsing or text generation, which makes it flexible for messy real-world queries.

WHAT TO SAY:

We use GLiNER only for initial entity suggestions. Then we apply rule-based filters to remove invalid spans like verbs or noise words. After that, we normalize the remaining text and check it against a product catalog. If there's no exact match, we use fuzzy string matching to correct typos by finding the closest valid product name. This ensures robust extraction even for messy real-world querie

---

Country extraction works in three steps. First GLiNER reads the sentence and suggests a country span based on grammatical context. We then validate that suggestion against our country catalog built from pycountry — 249 countries with full names, alpha codes, and common names. If it's an exact match, done. If it's a typo or shorthand like 'brzail' or 'pak', we run RapidFuzz fuzzy matching to find the closest real country. If GLiNER finds nothing at all, we fall back to regex — scanning for preposition patterns like 'from X' or 'in X' and running the same validation on whatever follow.

(read below if needed above is perfect)

How We Guarantee Perfect Country Extraction"

"When a user types a query, finding the country is a massive challenge because of typos, shorthand acronyms, and broken grammar. To solve this, our system runs the query through a linear, 4-Phase pipeline.

Phase 1: The AI Suggestion (GLiNER)

First, we ask our AI (GLiNER) to read the raw sentence. GLiNER uses bidirectional transformers to look at the grammar. If it sees the pattern *"buy sugar from X"*, it will suggest that **"X"** is a country.
But we do **not** trust the AI blindly. The AI only gives us a raw, often misspelled string like `"brazl"`. So, we take the AI's suggestion and force it through Phase 2.

Phase 2: The Validation Gauntlet (`_resolve_country`)

Every single suggestion made by the AI must survive this 4-step Validation Gauntlet. Its job is to mathematically map the AI's messy guess to an official database country, or reject it entirely.

1. **Check 1 (The Noise Filter):** The system first checks a banned word list. Did the AI accidentally extract a word like `"global"`, `"international"`, or `"countries"`? If yes, reject it instantly.
2. **Check 2 (The Slang Dictionary):** The system checks our custom trade dictionary. Did the AI extract shorthand like `"uae"` or `"s korea"`? If yes, it translates them to `"United Arab Emirates"` and `"South Korea"`, and the process **stops successfully.**
3. **Check 3 (Exact Match):** The system asks the database: *"Is this an exact spelling of a real country?"* (e.g. `"pakistan"`). If yes, it capitalizes it to `"Pakistan"` and the process **stops successfully.**
4. **Check 4 (The RapidFuzz Math):** If it's a typo, the system runs RapidFuzz. It mathematically calculates the similarity between the AI's guess (`"brazl"`) and the official catalog (`"Brazil"`). If the score is higher than our strict 85% threshold, the system fixes the typo and the process **stops successfully.**

If the AI's guess fails all 4 checks in this gauntlet, the suggestion is destroyed, and the system activates our Fallback Recoveries.

Phase 3: The Grammar Recovery Fallback

If the AI completely failed to find a country, or the Validation Gauntlet destroyed its guess, the system drops down to Phase 3.
We ditch the AI and use hardcoded Regex rules. The system scans the raw text specifically hunting for prepositions (`from`, `frm`, `in`). If it spots one, it blindly extracts the very next word (e.g. spotting `"frm chnia"` and extracting `"chnia"`).

It then takes `"chnia"` and sends it **back up to Phase 2** to run the Validation Gauntlet. The Gauntlet runs the RapidFuzz math, realizes `"chnia"` is a 90% match for `"China"`, fixes the typo, and the process **stops successfully.**

Phase 4: The Brute Force Recovery

If Phase 3 failed because the user didn't type a preposition at all (e.g. *"dextrose brazil"*), we drop to the ultimate safety net.

The system breaks the entire query into individual words. It scans every single word that is longer than 3 letters and asks the database: *"Is this an EXACT match for a country?"* We explicitly disable fuzzy math in this final layer to prevent the system from accidentally fuzzy-matching an English word (like `"find"`) to a country (like `"Finland"`). If it finds an exact match, the process **stops successfully.**"

How We Guarantee Perfect Product Extraction"

"Extracting a product from a natural language query is extremely difficult because B2B users search for tens of thousands of obscure chemicals and agricultural goods, often riddled with typos and bad grammar. To solve this, we don't rely on a single model. We run a 5-Phase pipeline that combines AI extraction with deterministic safety nets.

Phase 1: The AI Suggestion (GLiNER)

We start by feeding the raw, unedited query to our AI (GLiNER). We prompt it with a variety of labels like `"product"` and `"commodity"`. Because GLiNER uses bidirectional transformers, it reads the grammar of the sentence. If a user types *"urgently need to buy Unobtanium today"*, GLiNER doesn't need to know what "Unobtanium" is. It knows the noun sitting after the word "buy" is the commodity, so it extracts it.

But just like with countries, we **never** trust the AI blindly. We take its raw guess and force it through Phase 2.

Phase 2: The Rejection Shields

GLiNER occasionally gets confused by terrible grammar and extracts the wrong span of text. To prevent catastrophic search failures, the AI's guess must survive two hardcoded Rejection Shields:

1. **Shield A (The Verb Shield):** Sometimes the AI accidentally sweeps an action verb into the product (e.g. guessing `"exporting sugar"`). The engine immediately checks if the text starts with known verbs (`import`, `export`, `buy`, `sell`). If it does, the guess is destroyed.
2. **Shield B (The Country Shield):** Sometimes the AI gets confused by a misspelled country and guesses it is a product. We take the AI's guess and fire it into our **Country Resolver**. If the resolver does the math and confirms the text is actually a country, the guess is destroyed.

Phase 3: The Complete Failure Recovery (Stop-Word Stripper)

If the AI failed completely, or if Phase 2 destroyed its guess, our system does not crash. It activates the ultimate safety net.

We ditch the AI and use pure **mathematical subtraction**. The engine takes the raw query and aggressively deletes every single known trade word (`"buy"`, `"sell"`, `"cheap"`, `"urgent"`, `"from"`, `"in"`). Whatever words survive this massive deletion are officially crowned as the product. The system guarantees we *always* have a product keyword.

Phase 4: The Cleaning Sweepers

We now have our core product (either from the AI or from Phase 3). But it might still have "junk" attached to it. We run it through two sweepers:

1. **The Preposition Sweeper:** We use Regex to hunt for dangling bridge words like `"from"` or `"of"` that accidentally got stuck inside the product string, and surgically delete them.
2. **The RapidFuzz Noise Stripper:** We run fuzzy-math against a banned list of trade slang. If a word sitting next to the product is an 80% mathematical match for a noise word like `"quality"` or `"premium"`, we erase it.

Phase 5: The Catalog Spell Checker

We finally have an isolated, perfectly clean product string. But there is one final fatal flaw: the user might have misspelled it (e.g., `"suggar"`).

As the absolute final step, the system splits the product string into individual words and runs RapidFuzz against our official, pre-loaded Postgres Database Catalog. It mathematically compares the user's typo (`"suggar"`) against the database. Because it is a >60% match for the official term `"Sugar"`, the engine fixes the spelling.

Only then does the pipeline stop, guaranteeing that the final string being sent to the database matches your official schema flawlessly!"

11:06 PM